

Computer Science Department
Software Engineering & Business Analysis

Bachelor's Thesis

Capstone Project Template

Longer Subtitle

Marko-Oleksandr Hlibovytskyy

Supervisor

KSE, Oleksandr Syrotiuk, PhD, osyrotiuk@kse.org.ua

Expert

Company, Expert Name, expert@domain.ua

Submission date

14 August 2025

TODOS

Academic Integrity Statement

I, undersigned, hereby declare that this capstone project is the result of my own work.

- All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
- No part of this project has been submitted previously for academic credit in this or any other institution.
- All code, diagrams, and third-party materials are either my original work or are used with permission and properly referenced.
- I have not engaged in plagiarism or any form of academic dishonesty.
- Any assistance received (e.g. from peers, tutors, or online forums) is acknowledged in the acknowledgements section.

I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date Kyiv, 03.06.2026

Signature _____

Contents

Acknowledgements	1
Abstract	2
1 Introduction	3
1.1 Background and motivation	3
1.2 Problem statement	3
1.3 Research objective and scope	3
1.4 Aim and Objectives	4
1.5 Methodology	5
1.6 Structure of the thesis	5
2 Research	7
2.1 Research questions and initial functional requirements	7
2.2 Candidate solutions	8
2.2.1 Unity Asset Store map generators	8
2.2.2 Non-Unity generators	15
2.3 Candidate evaluation	17
2.4 Gaps and missing features	19
2.5 Video game examples	20
2.5.1 Games with hexagonal cell grid maps	20
3 Design	25
3.1 Functional and non-functional requirements	25
3.2 Architecture	26
3.2.1 Framework-specific terminology	27
3.3 System diagrams	29
3.4 Technology stack	31
3.5 Security	32
4 Implementation	33
4.1 Methodology	33
4.2 Prototyping	33
4.3 Project standards	33
4.3.1 Core component implementation	33
4.4 Testing	33
4.5 Performance bottlenecks and optimizations	33
4.6 Deployment	33
4.7 Documentation	33
5 Validation	34
5.1 Testing approaches for functional requirements	34
5.2 Testing approaches	34
5.3 Test cases	34
5.4 System limitations	34
6 Conclusion	35
6.1 Thesis summary	35
6.2 Alignment with objectives	35

6.3 Challenges	35
6.4 Limitations	35
6.5 Future work	35
Glossary	36

Acknowledgements

Abstract

1 | Introduction

1.1 Background and motivation

Game studios face a persistent challenge: manually created content is expensive, time-consuming, and finite, while players expect large, expansive worlds and replayability.

Although, game developers have found a way to compensate for this problem. Procedural generation addresses this issue by allowing developers to create content via algorithms rather than relying solely on asset designers. Most obvious cases are generated game worlds.

A few prominent examples:

- Minecraft
- Civilization series
- Heroes of Might and Magic series (HoMM)
- Age of Wonders
- Hades 1 & 2
- And many more

1.2 Problem statement

In practice, procedural generation has solved only part of the problem. After repeated play, players may begin to recognize generation patterns. The moment the player gains a working understanding of the generation system is pivotal to their perception of the game. The interest that draws the players in weakens as the sense of novelty fades.

Procedural generation increases replayability, but it does not guarantee meaningful variety. Problems leading to predictable generation can be succinctly summarized into two:



Flatness: lack of layers, interactions, and relationships between features.



Monotony: insufficient or little variance in generation caused by weak randomization or a limited set of designer-made features.

1.3 Research objective and scope

The scope of this thesis was deliberately limited to procedural generation of maps using grids made of hexagons. This limitation exists to avoid an overly broad and vague treat-

ment of procedural generation. As mentioned above, procedural generation covers many possible applications, therefore attempting to support all of them would make the project difficult to explain, design, and evaluate.

This thesis narrows procedural generation to a specific technical setting – hexagonal game maps generated inside Unity using graph rewriting. The detailed reasoning behind these choices is discussed later in this section and other chapters, but they are introduced here because they define the object of research and the boundaries of the prototype.

Object of research – procedural generation of game maps composed of hexagonal cells, implemented in Unity through graph rewriting.

In this proof-of-concept a graph is defined as:

- Vertex – hexagonal cell
- Vertices connected via edges are considered neighbors
- Graph structure – grid gapless hexagonal tiles.

Generation process:

1. Generation rules look for patterns defined by designers
2. If pattern matches exist, the graph is rewritten according to the designer-defined active rule

Reasoning behind hexagonal cells:

1. Enough examples exist for evaluation in strategy/tactical/puzzle games.
2. Map type restriction keeps the topic from becoming too broad.
3. Hexagon-shaped cells allow for more complexity than square cells, which are more common in video games.
4. Player cell-to-cell movement is equidistant in all directions to neighboring cells, creating more even movement than square cells.
5. Hexagonal cells require a more complex implementation than square cells, making future support for square or other cell types simpler than implementing a new framework from the ground up.

In addition, this limited scope gives clear scenarios to benchmark against. Maps made up of hexagons are not that common in video games, mostly appearing in strategy/tactical, puzzle genres of games. For the most part, strategy games will be used as benchmarks.

1.4 Aim and Objectives

The aim of this thesis is to design and implement a proof-of-concept procedural generation framework for hexagonal game maps in Unity.

Objectives:

1. Analyze existing uses of procedural generation in games.
2. Identify causes of predictable or repetitive map generation.
3. Define requirements for a designer-friendly generation framework.
4. Design a configurable model for hexagonal map generation.
5. Implement the framework as a Unity prototype.

6. Validate the prototype through generated map examples for example game concepts.

The framework aims to expose designers to features enabling them to configure generation in a flexible way. Generation set up should happen entirely within the engine editor, requiring no code. As a result, designers should be empowered to create more meaningful and varied generation without the prerequisite of having a lot of technical knowledge.

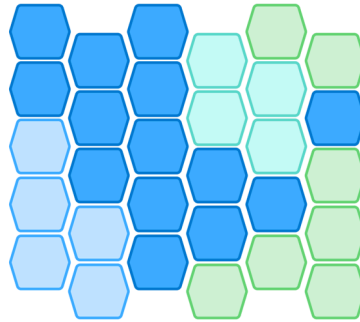


Figure 1: Sample square hexagonal grid



Framework uses a hexagonal cell grid to build game worlds



Aim: proof-of-concept Unity framework that provides game designers with flexibility in generation design



Main context for benchmarking – strategy games

1.5 Methodology

This thesis follows an applied engineering approach.

At first, existing procedural generation approaches and relevant game examples are reviewed. Following that, requirements are defined for a configurable proof-of-concept framework. After requirements are defined, the system is designed and built in Unity. Lastly, the prototype is evaluated using example game concepts and comparison against existing games.

1.6 Structure of the thesis

- **Chapter 2** – Research of related work and approaches to procedural generation.
- **Chapter 3** – design of the proposed framework.

- **Chapter 4** – implementation in Unity.
- **Chapter 5** – validation of the framework through selected generation scenarios.
- **Chapter 6** – summary of the results and thesis.

2 | Research

2.1 Research questions and initial functional requirements

In this chapter, existing procedural generation tools, frameworks, and game examples will be examined in order to identify existing capabilities and which gaps are left open. The research focuses on map generation workflows, designer control, tool accessibility, and support for hexagonal grid-based generation. Observations from the research will be used to derive requirements for the prototype framework.

Research questions:

1. What existing procedural generation tools and frameworks are available for game map generation, especially in Unity?
2. How do existing tools balance designer control, ease of use, flexibility, and technical complexity?
3. To what extent do existing tools support hexagonal grid-based map generation?
4. What gaps in existing tools justify the development of a focused Unity framework for hexagonal map generation using graph rewriting?
5. How do existing video games use procedural generation for map variety and feature-richness and what generation options are exposed to players or designers?

Based on the thesis scope defined in Section 1 and the following research, the framework should satisfy the following functional requirements:

ID	Requirement
1	The framework shall produce a game map as a graph of hexagonal cells.
2	The framework shall allow designers to define map generation rules based on global or local cell patterns.
3	The framework shall allow designers to define what pattern matches are considered valid candidates for rule transformations.
4	The framework shall allow rules with matches that are considered valid to transform the grid during map generation.
5	The framework shall expose generation configuration inside the Unity editor Inspector window.
6	The framework shall allow map generation without requiring designers to write little to no code.
7	The framework shall support repeatable and stable generation through configurable parameters and seeds, which will guarantee a deterministic result across configurations.
8	The framework shall provide visual feedback for generated hexagonal maps inside the Unity editor scene view.

Table 1: Functional requirements for the proposed framework

Important note: Although hexagonal grids are common in strategy/tactical, puzzle games, the proof-of-concept framework is not limited strictly to these genres. Genre compatibility serves us in the context of the defined strategy-centric scope rather than being a strict requirement.

2.2 Candidate solutions

The following items will be inspected in the reviewed solutions:

- Flexibility in generation setup and settings.
- Designer-friendliness/code use for setup.
- Unity integration.
- Cost and accessibility.
- Learning curve.
- How wide or narrow the scope of existing solutions is.

Since the framework will be developed using the Unity engine, solution exploration will begin with the Unity Asset Store.

Packages were found by searching “procedural generation”, “procedural map generation”, “procedural world generation” and other search terms with minimal changes. Most of the reviewed packages were categorized under **Tools/Terrain**, **Tools/Level Design** or **Tools/Utilities**

Valid candidates for evaluation found in the asset store:

1. DunGen
2. RoomGen - Procedural Generator
3. Edgar Pro - Procedural Dungeon Generator
4. Dungeon Architect
5. Procedural Generation Grid

2.2.1 Unity Asset Store map generators

Important note: Unity Asset Store packages often provide limited public documentation before purchase. As a result, the evaluation relies on store descriptions, available documentation, public reviews, and community discussions where official documentation is unavailable.

The 5 listed candidates can be categorized into 2 categories:

Category	Reviewed solutions
Room/prefab-based dungeon generators	DunGen, RoomGen, Edgar Pro, Procedural Generation Grid
Broad procedural generation frameworks	Dungeon Architect

Table 2: Categories of reviewed procedural generation solutions

To make the comparison easier to follow, candidates will be compared together by category. First, candidates will be reviewed according to categories listed in the previous subsection. Then, in the Candidate evaluation subsection, packages will be compared to one another.

Room/prefab-based dungeon generators: 1. DunGen:

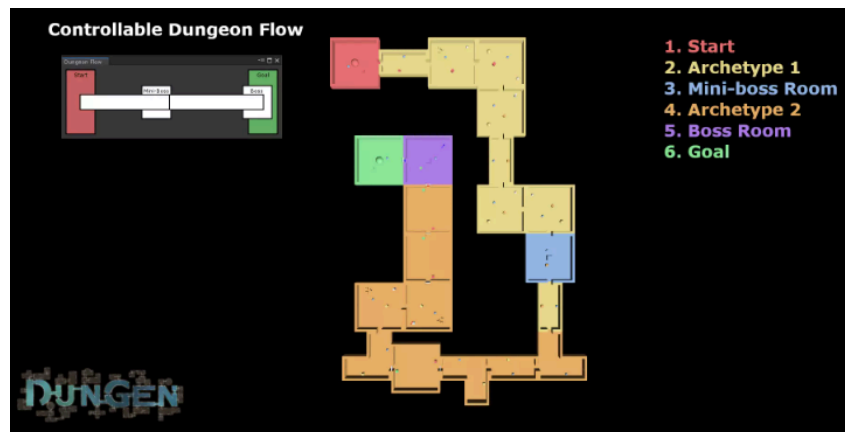


Figure 2: DunGen map view during the generation process

1. Flexibility in setup and settings:
 - Node-based room relationship setup allows for easy generation customization. User defines dungeon flows, tile sets, branching paths, locks, keys, object placement rules.
 - Setup focuses on dungeon-style layouts composed of room prefabs.
 - Uses a visual dungeon flow graph.
 - Archetypes consist of one or more tile sets and are used to define behavior for dungeon sections (branching, path straightening, branch caps and branch pruning and so on).
2. Designer-friendliness:
 - The package is designed around Unity editor assets and visual configuration. Little to no coding is required for basic use.
 - Coding is required when the user wants behavior that cannot be expressed through its inspector assets and built-in rules.
3. Unity integration:
 - Developed as a Unity Asset Store package
 - Workflow closely tied to Unity concepts.
4. Cost and accessibility:
 - Cost: listed for **\$79.99**.
 - Listed as a paid Asset Store package.
 - More accessible than large professional tools, which will be considered later, but still commercial.
5. Learning curve:
 - Basic setup is approachable, however dungeon flow concepts and tile rules require some learning.

6. Scope of the solution:

- The package primarily focuses on procedural dungeon generation using connected rooms/tile prefabs.
- The scope is relatively narrow. The package aims to provide designers with a flexible dungeon generator, but it also includes assets for generation.
- Markets itself as a one-stop-shop for dungeon generation since it includes a generation framework and assets for map generation.

2. RoomGen:

1. Flexibility in setup and settings:

- RoomGen focuses on generating rooms, buildings, modular interiors, dungeons and dense landscapes from modular tiles and presets which the user provides.
- Setup is based around configurable presets. A designer creates a preset, assigns tiles and decorative objects, adjusts generation settings, and generates the result inside the Unity editor.
- Package allows depth in dungeons via multi-floor setups, where each floor can use different presets and settings.
- RoomGen exposes many customization options for map generation like tile weights, object placement probabilities, object spacing and other spacial placement settings.
- It does not expose an explicit graph-based workflow.



Figure 3: Dungeon screenshot from RoomGen Unity Asset store page.



Figure 4: Landscape screenshot from RoomGen Unity Asset store page.

1. Designer-friendliness:
 - Package explicitly designed around ease of use. Package advertises a workflow consisting of preset creation, dragging and dropping tiles and objects within the Unity editor and generating inside the Unity editor.
 - Basic use does not require coding. Most features are set up inside the Unity editor.
 - Code required when using RoomGen at runtime. If designers choose to generate dungeons during gameplay, then scripts need to be used to trigger map generation.
2. Unity integration:
 - Developed as a Unity Asset Store package
 - Workflow closely tied to Unity concepts.
3. Cost and accessibility:
 - Cost: **\$39.99**.
 - Listed as a paid Asset Store package.
 - Compared to other candidate solutions and larger tools which will be inspected later, RoomGen is relatively accessible in cost and ease-of-use.
4. Learning curve:
 - Basic workflow for map generation setup is simple because it is based on presets, modular tiles and object placement rules.
 - Without the use of scripts, maps can only be generated in the Unity editor. For dynamic runtime generation, coding is required.
5. Scope of the solution:
 - Broader scope than pure dungeon map generation. Aside from dungeon generation, rooms, buildings and landscapes can be generated.
 - The package is centered around modular room, dungeon, building, landscape creation and variation. Allows designers to create varied sets of assets from prefabs.

3. Edgar Pro:

1. Flexibility in setup and settings:

- Edgar Pro focuses on generating 2D dungeons.
- Its generation model consists of handlade room templates and a graph containing the level structure description.
- The level graph is the central structure. Nodes represent rooms and edges represent necessary connections between rooms. The graph describes which rooms must exist and how rooms should be interconnected.
- Rooms can be connected directly or via corridors.
- The level graph provides explicit control over dungeon structure.
- The package stitches rooms together, but does not modify the rooms themselves.

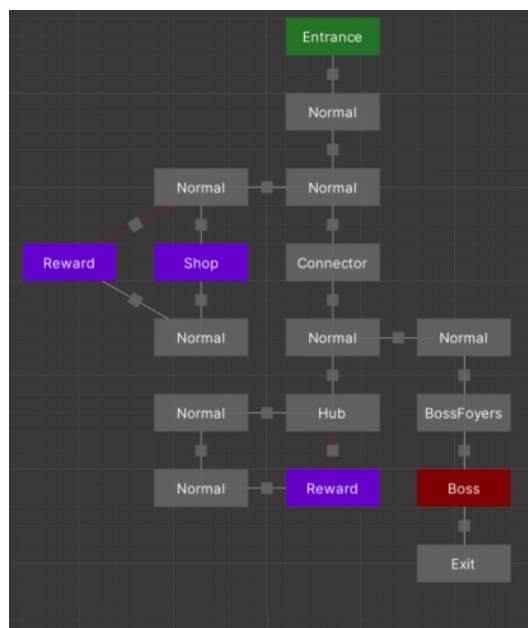


Figure 5: Level graph screenshot from Edgar Pro Unity Asset store page.

1. Designer-friendliness:

- Visual workflows inside the Unity editor are used to configure level graphs and room templates.
- Basic generation setup is performed entirely in the Unity editor.

2. Unity integration:

- Developed as a Unity Asset Store package.
- Workflow closely tied to Unity concepts.

3. Cost and accessibility:

- Cost: **\$55**.
- Edgar Pro is a paid Unity Asset Store package.

4. Learning curve:

- Edgar Pro makes basic setup approachable. However, a lot of details need to be set up for dungeon generation. For example, room templates, tilemaps, room outlines, door positions, corridor templates, etc.
- The level graph facilitates flexible setup, but it requires the user to think in terms of room relationships to generate interesting layouts.

5. Scope of the solution:

- Edgar Pro is narrow in scope, but provides user with deep flexibility in its niche.

Procedural Generation Grid:

1. Flexibility in setup and settings:

- Procedural Generation Grid uses grids to define fields and paths for procedural generation.
- First grid size and prefabs are defined, then the designer sets up generation rules.
- Generation rules handle mutation of the grid via grid modifiers. Designers define rule conditions which have to be true for grid mutation to take place.
- Includes a Node Graph system, allowing designers to combine multiple field setups into a larger generation workflow. This makes it possible to compose several grid generation stages into a more complex map generator.



Figure 6: Generated dungeon screenshot from Procedural Generation Grid Unity Asset store page.

1. Designer-friendliness:

- Package provides visual scripting-like tools for defining its generation workflows. Basic setup is done entirely within the Unity editor with the help of the package's visual tools.
- Coding is not required for basic use, but designers are still required to understand rule-based generation, grid modifiers, and other concepts to set up map generation.

2. Unity integration:

- Developed as a Unity Asset Store package.
- Workflow closely tied to Unity concepts.

3. Cost and accessibility:

- Cost: **\$45.99.**
- Procedural Generation Grid is a paid Unity Asset Store package.
- Price falls within expected range for narrow scope packages which is based on previously reviewed candidates.

4. Learning curve:

- For the most part the package avoids coding for its workflows. Despite that, the package requires the designer to learn the concepts in its generation model to utilize the flexibility of the map generator.

5. Scope of the solution:

- Scope is closer to this thesis as it utilizes grid cells and rule-based generation.
- Fairly narrow scope, focusing on dungeon-like generation and prefab placement.

Broad procedural generation frameworks:

Dungeon Architect:

1. Flexibility in setup and settings:

- The package includes several builder types like, including Grid Flow Builder, Snap Builder, Grid Builder, Snap Grid Flow Builder and City Builder.
- The Grid Flow Builder allows designers to create node based dungeon flows with cyclic paths, a variety of ways to connect rooms like teleporters and one-way doors, tilemap-based layouts.
- Dungeon Architect provides very high flexibility. Its flexibility comes from supporting many different procedural workflows.

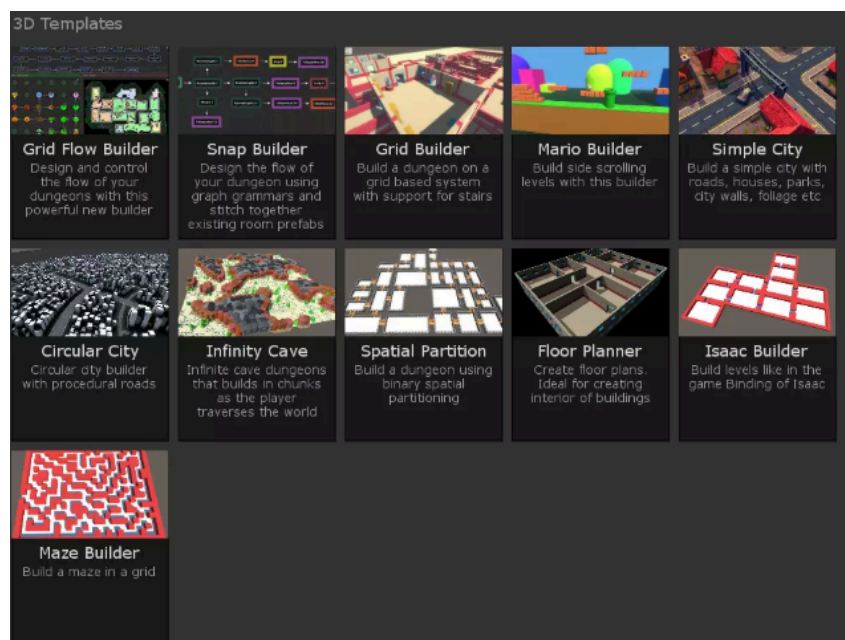


Figure 7: 3D templates screenshot from Dungeon Architect Unity Asset store page.

1. Designer-friendliness:

- The package provides visual Unity editor tools for most of its systems.
- Package comes with many specialized systems
- Coding required for advanced customization.

2. Unity integration:

- Developed as a Unity Asset Store package.
- Workflow closely tied to Unity concepts.

3. Cost and accessibility:

- Cost: **\$300**.
- Dungeon Architect is a paid Unity Asset Store package.
- Comes at a significantly higher price than previous packages but is more feature-rich. At this price point, the package is less accessible for small teams, hobbyists and students.

4. Learning curve:

- Due to the sheer amount of features and options presented to the designer, the learning curve is high.
- Designers have to get acquainted with generation workflows and different builders offered by the package.

5. Scope of the solution:

- Dungeon Architect's scope is quite wide and feature-rich.
- This package could be considered a full procedural generation suite, rather than a map, dungeon or landscape builder.
- For this thesis, Dungeon Architect is relevant because it demonstrates mature visual procedural generation in Unity. This package establishes what a wider scope includes.

2.2.2 Non-Unity generators

Unreal Engine 5:

Unreal Engine Procedural Content Generation Framework:

1. Flexibility in setup and settings:

- Unreal Engine's PCG Framework is a built-in procedural generation toolset for creating procedural content inside Unreal Engine.
- Utilizes procedural node graphs that can generate content in the editor and at runtime.
- Flexibility stems from a general-purpose graph workflow. Designers express generation rules through connected nodes in a visual graph.
- Connections between nodes define order and relationships between operations.
- Node operations cover a wide array of features. For example, nodes can generate sets of points, use attributes for filtering, transform points in numerous ways, spawn actors or static meshes and combine these operations in a wide range of scenarios.
- This allows the framework to support a wide range of procedural tasks, such as environment population, foliage placement, biome generation, object scattering, spline-based generation, and runtime procedural content.
- This framework is tied to Unreal Engine's ecosystem and does not support Unity.

2. Designer-friendliness:

- PCG provides visual graph tools, making it accessible without traditional scripting for many procedural tasks.
- However, designers need to be familiar with Unreal Engine 5, PCG graphs, runtime generation behavior, spatial constraints and the other many concepts that the generation features are composed of.

3. Unity integration:
 - Unreal PCG is not integrated with Unity. It is part of Unreal Engine.
 - Because this thesis targets a Unity-based framework, Unreal PCG is useful as a point of comparison but not as a direct candidate solution.
4. Cost and accessibility:
 - Free.
 - The framework is available inside Unreal Engine 5.
 - Access to the framework comes with adopting Unreal Engine 5.
5. Learning curve:
 - The learning curve is significant for users who are not already familiar with Unreal Engine 5.
 - For teams already using Unreal, PCG is powerful and accessible. In the context of this thesis, it is used as a point of reference.
6. Scope of the solution:
 - Unreal PCG has a broad scope, providing a solution for many procedural generation-centric problems.
 - PCG is a demonstration of what mature procedural generation tooling can provide.

Houdini FX:

Houdini FX and Houdini Engine:

1. Flexibility in setup and settings:
 - Houdini FX is a professional procedural content creation tool based on node networks.
 - Houdini possesses the most expansive set of feature out of all the candidates. It can be used for procedural modeling, terrain generation, destruction, fluids, cloth, particles, simulations, asset generation, and game content pipelines.
 - Houdini workflows can be packaged as Houdini Digital Assets, which expose selected parameters to artists or designers.
 - Through the Houdini Engine, digital assets created inside the engine can be loaded into game engines such as Unity and Unreal Engine, allowing procedural controls to be used inside the engine editors.
 - This makes Houdini one of most flexible reviewed solution, but its flexibility comes from being a full procedural Digital Content Creation and technical art platform rather than a focused game asset generator, let alone a map or dungeon generator.
2. Designer-friendliness:
 - Houdini can expose simplified controls through Houdini Digital Assets, but creating those assets requires significant technical knowledge.
 - To create a procedural system, a technical artist must understand Houdini's node workflow, geometry processing, parameters, asset packaging, and possibly scripting.
 - Therefore Houdini can be designer-friendly for the end user of a prepared asset, but not necessarily for the person building the generator asset itself.
3. Unity integration:

- Houdini Engine provides a Unity plugin that allows Houdini Digital Assets to be used inside the Unity editor.
 - The Unity plug-in does not replace Houdini itself. A Houdini Engine, Houdini Core, or Houdini FX license is required to run procedural assets through the plugin.
4. Cost and accessibility:
 - Houdini has free or lower-cost options for non-commercial or indie use, but production use depends on SideFX licensing.
 - Compared with Unity Asset Store tools, Houdini is less accessible for students or small teams if the goal is only a focused hexagonal map generator.
 - It also requires learning an additional professional tool outside Unity.
 5. Learning curve:
 - The learning curve is very high compared with the reviewed Unity packages.
 - Houdini is powerful because it exposes low level procedural generation control, but this also means users must understand Houdini Engine concepts.
 6. Scope of the solution:
 - Houdini has the broadest scope of all reviewed solutions.
 - It is capable of producing many types of procedural content, but this makes it excessive for the kinds of relatively specific problems that most reviewed Unity packages solve.

2.3 Candidate evaluation

Unity packages:

Most reviewed candidate Uinty packages provided solutions for reasonably narrow-scoped problems, with the exception of Dungeon Architect. DunGen, RoomGen, Edgar Pro, Procedural Generation Grid are focused on allowing game designers to create a map/dungeon for their game. Some offer slightly deeper functionality like multi-layer dungeons or object generation. The scope of these packages loosely resembles the scope of the proof-of-concept framework of this thesis.

Most notably, a relationship can be traced in setup generation among the packages. Despite all of the candidates being little-to-no-code solutions, designers still face the prerequisite of learning generation models and which components implement it. Coding is traded for workflows with visual setup inside the Unity editor, but learning the system is required in every case to a varying degree. For the game designers, this means that they still need to familiarize with technical aspects of the system.

Dungeon Architect encompasses significantly more map generation options than the rest of the packages. While it has deep functionality, its feature-richness goes far beyond the scope of the proof-of-concept framework of this thesis. For that reason, it does not serve as a good comparison. But it is worth restating that Dungeon Architect actively uses node-based setup for map/dungeon generation with the ability to have directed graphs, cyclical sections and a plethora of unique ways to connect nodes. The graph structure Dungeon Architect Templates provide is quite flexible. Still, it is limited in expressing the structure hex cell grid generation needs.

Procedural Generation Grid possesses qualities that are very similar to what my requirements describe. Though it uses square cell grids, the generation model is conceptually similar. The designer provides prefabs the dungeon generator can use during generation. Then, generation rules are defined by the designer. In short, rules consist of defining how cells can be placed in relation to others. The generation model is flexible, making it a suitable example to compare against.

Explicit hexagonal cell support is not mentioned explicitly anywhere. Most packages outright do not support them.

Non-Unity solutions:

Unreal Engine PCG:

This framework has a broad scope, allowing for versatile generation and covering different use cases. It does not support Unity, so it is not a valid candidate for proof-of-concept implementation. But it serves as a great reference for a full suite of procedural generation tools.

It has the capability to generate hexagonal cell grids, but the functionality of the node operations can still fall short of a standalone solution aimed specifically at hex cell maps.

Unlike solutions offered by the Unity Asset Store, Unreal Engine's PCG required deeper fundamental knowledge of the game engine and generation tools.

Houdini FX:

Since Houdini is a Digital Content Creation and technical art platform, this solution has applications ranging outside editor tools and game asset generation. While its wide scope sets Houdini apart from other reviewed solutions, it also sets itself apart due to sheer complexity of the tool. Houdini Assets can be more user friendly and centered around a specific concern, the creation of said assets still requires a lot of technical knowledge. Because of Houdini's complexity and licence pricing, it is the least accessible solution out of all candidates.

Like Unreal Engine's PCG, Houdini FX is not a suitable platform for the creation of a proof-of-concept framework. The exact same reasons apply – the platform's feature set is far too broad for my requirements and the system is not accessible enough.

Solution	Platform	Main scope	Role in this thesis
DunGen	Unity	Dungeon / room-based generation	Narrow benchmark
Edgar Pro	Unity	Dungeon / level generation	Narrow benchmark
Dungeon Architect	Unity / Unreal	Broad dungeon and level generation framework	Strong but oversized benchmark
Procedural Generation Grid	Unity	Grid-based procedural generation	Closest Unity benchmark
Unreal PCG	Unreal Engine	General procedural content framework	Reference system
Houdini / Houdini Engine	External tool / Unity integration	General procedural asset generation	Reference system

Table 3: Positioning of reviewed procedural generation solutions

Solution	Unity integration	Hex grid support	Designer-facing workflow	No-code setup
DunGen	Yes	No / unclear	Yes	Partial
Edgar Pro	Yes	No / unclear	Yes	Partial
Dungeon Architect	Yes	Possible	Yes	Partial
Procedural Generation Grid	Yes	Partial / closest	Yes	Partial
Unreal PCG	No	Possible with custom work	Yes	Partial
Houdini Engine	Partial	Possible with custom work	Partial	No / technical

Table 4: Feature fit of reviewed tools against the thesis scope

2.4 Gaps and missing features

Some Unity packages contained node-based logic and workflows and some also used grids for prefab arrangement and map generation. Unity candidates provide flexible, relatively designer-friendly and relatively accessible solutions. But not a single one of them satisfies the requirements for the generation of a grid of gapless hexagonal tiles.

Unreal Engine PCG and Houdini FX offer capabilities to implement similar generation models. For the most part, they do not offer Unity support, require significantly more technical knowledge and are less accessible, with the exception of Unreal Engine's PCG being free. These solutions have a significantly smaller gap in missing features for the

satisfaction of my requirements, but PCG and Houdini will serve only as reference in this thesis.

Capability	Found in reviewed tools?	Research gap
First-class hexagonal cell grid generation	No	Reviewed tools do not directly target the exact map type used in this thesis.
Unity native hex cell grid map generation workflow	No / limited	Unity tools mostly focus on dungeons, rooms, terrain, or general grid workflows.
Designer defined graph rewriting rules	No	Reviewed tools do not expose generation as designer defined pattern matching and graph transformation.
Focused hex cell grid map scope	No	Broader systems can theoretically support this, but they are not designed around this specific use case.
No-code configuration inside Unity Inspector	Partial	Some tools expose settings visually, but not for configurable hex cell graph rewriting.
Deterministic hex-map generation for comparison	Partial	Repeatable generation may exist, but not in combination with the specific hex-grid rule system proposed here.

Table 5: Missing features related to the proposed hexagonal map generation scope

2.5 Video game examples

Understanding how games use procedural generation is vital to understanding what solutions developers may run into. The video games that will be review below serve as design benchmarks rather than technical competitors. The point is not to answer questions like “Which game has the best generator?”

For this section, the following questions will be used:

1. What do successful games expose to users?
2. What kinds of variety do their generators create?
3. Do generated features interact meaningfully?
4. Do any of them use hex cells in a way relevant to this thesis?

2.5.1 Games with hexagonal cell grid maps

Civilization V:

Civilization V is a turn-based 4X strategy game and one of the most relevant examples for this thesis because its main game map is built from hexagonal tiles. Civilization V introduced hexagonal tiles to the series. This makes the game a useful reference point for evaluating how hexagonal maps support:

1. strategic movement
2. expansion
3. terrain evaluation
4. replayability

Civilization V provides a notable assortment of player-facing setup options. It does not offer low level control over generation. Before starting a match, players can choose parameters such as map type, map size, world age, temperature, rainfall, sea level and resource abundance. These options allow the player to influence the generated world without needing to understand the internal generation algorithm.

The generator produces variety in generated maps with the help of interactions of several map features. Terrain, water, mountains, resources, natural wonders, city-state placement, civilization starting positions and expansion space availability all affect the gameplay value of a generated map. This depicts that map variety isn't simply tied to changes like map size, shape of the land or adding more features. Generation variety stems from how generated features are arranged and interact with one another. Civilization V game mechanics utilize different possible arrangements of features to create different gameplay scenarios for players, inciting replayability.

A little more about the gameplay – power dynamics between civilizations can vary, sometimes greatly, because of the map settings. Especially if one or several of the players may get lucky with luxury resource and natural wonder generation. Generation of these strategic points of interest may depend on one another and can be less or more likely depending on the settings. On top of that, the individual abilities civilizations have may allow them to harness such an advantage gained from the map generation to an even further extent.

Civilization V is a prominent example as its hexagonal map is not only a visual grid. Hex cells are central to movement, combat positioning, city placement, resource access, and territorial expansion. The hex grid is part of the game logic rather than just a display format. This supports the thesis assumption that hexagonal cells are a suitable structure for procedural map generation that creates flexibility in design and gameplay.

However, the game does not provide a reusable generation framework. Its generator is tightly connected to the rules and specific balance of the game. Players can configure high level parameters, but they cannot define their own generation rules or graph transformations.

In the context of this thesis, Civilization V serves as a design benchmark as it demonstrates the value of hexagonal maps and interacting generated features, but it does not provide the kind of designer configurable Unity framework proposed in this work.



Figure 8: Civilization V gameplay screenshot.

Age of Wonders 4:

Age of Wonders 4 (AoW 4) is a turn-based 4X strategy game with tactical combat. It is relevant to this thesis because its strategic map uses hexagonal tiles and because its world generation exposes a large number of high level configuration options to the player.

In AoW 4, generated maps and gameplay scenarios are called realms. AoW 4 allows players to shape the generated world through realm traits. These traits can affect geography, climate, inhabitants, special world conditions, and the presence of powerful factions or enemies. The resulting generated map is not only varied spatially, but also thematically and mechanically.

The game creates variety in generated maps through interacting systems. Geography affects movement, expansion, and access to resources. Climate and terrain influence the usability and strategic value of different regions. Inhabitants, hostile enemies, free cities which the player can claim, and special realm modifiers can change how the player approaches the main game mechanics – exploration, combat, expansion, and diplomacy. Replayability is not created merely by rearranging terrain, enemy placement, free city placement and other features. The game combines different map layout with gameplay systems that respond to the features of the generated world.

The strength of AoW 4 as an example is that its generation is configurable while remaining easy to understand for players. Realm setup options allow players to influence the type of experience they want without making low level generation changes. This is relevant to the proposed framework for the reason that it reinforces the value of exposing generation control through readable, designer facing options.

Hexagonal cells are also important to the gameplay of AoW 4. The hex grid structures movement, exploration, territorial expansion, army positioning, and tactical decision making. Similar to Civilization V, the hexagonal map is not merely a visual representation. Hex tiles directly influence the game's strategic logic and affect how players understand and influence power dynamics between them and their rivals.

Age of Wonders 4 does not provide a reusable procedural generation framework, like Civilization V. Its generation system is deeply tied to the game setting, faction design and various systems. Players have high level control over map properties, but they cannot define custom generation rules or reusable map generation logic.

In the context of this thesis, Age of Wonders 4 serves as a design benchmark for configurable, feature-rich hexagonal map generation. It demonstrates that meaningful procedural variety can come from the interaction of geography, world traits, factions, terrain, and gameplay rules.



Figure 9: Age of Wonders 4 gameplay screenshot.

Heroes of Might and Magic III:

Heroes of Might and Magic III (HoMM III) is a turn-based strategy game with exploration, resource collection, town development, and tactical combat. It serves this thesis as a comparison point since it separates two different grid use cases:

1. overworld exploration takes place on a square tile adventure map
2. combat takes place on a hexagonal combat grid

The game shows that grid choice can depend on the type of interaction being represented. On the adventure map, the player navigates terrain, visits buildings, collects resources and interacts with various map objects. These objects are often visually and spatially aligned with a square map structure. Square tiles have the upside of making the adventure map readable and practical for placing buildings, roads, obstacles, and other designer-made objects.

Combat, however, uses a hexagonal grid. Hexagonal cells suit this context well. Tactical positioning, movement range, and attack distance are central to gameplay. As stated in Section 1, hexagonal tiles provide more neighboring directions than square tiles without introducing diagonal distance inconsistencies. In the context of HoMM III, hexagonal tiles facilitate more complexity and tactical expression in the combat system. Compared with square cells, hexagons avoid diagonal movement inconsistencies and provide uniform distances to neighboring tiles. This makes them useful for battle systems where relative position and distance need to be clear.

This distinction is relevant to the thesis as it shows that hexagonal cells are not automatically the best choice for every game map. They are particularly useful when movement, adjacency, and tactical positioning are important for the quality of gameplay. At the same

time, square grids may be easier for arranging buildings, rectangular objects, roads, and other various map structures. This supports the thesis decision to focus specifically on hexagonal game maps where the hex grid has gameplay meaning, rather than treating hexes as a universal end-all-be-all replacement for all map types.

In the context of this thesis, HoMM III serves as a contrast benchmark. It shows a successful strategy game that uses hexagonal cells for tactical combat, but not for its main adventure map. This reinforces the idea that the proposed framework should be evaluated in scenarios where hexagonal cells provide clear design value, such as movement, positioning, neighborhood relationships, and tactical or strategic map logic.



Figure 10: Heroes of Might and Magic III combat gameplay screenshot.

Example	Relevant observation	Value for this thesis
Civilization V	High-level map settings influence terrain, resources, climate, sea level, world shape, and starting conditions.	Shows how player-facing generation settings can create varied strategic scenarios on a hexagonal map.
Age of Wonders 4	Realm traits expose high-level control over geography, climate, inhabitants, world conditions, and major powers.	Shows how layered generation settings can create thematic and mechanical variety without exposing low-level algorithms.
Heroes of Might and Magic III	The game uses a square adventure map but switches to a hexagonal grid for tactical combat.	Shows that hexagonal cells are especially valuable when movement, adjacency, positioning, and attack distance are central to gameplay.

Table 6: Value of selected game examples for the thesis

3 | Design

3.1 Functional and non-functional requirements

The research section identified a gap for a focused Unity framework for designer-configurable hexagonal map generation. This section translates that gap into functional and non-functional requirements for the proposed proof-of-concept.

Functional requirements describe the behavior the framework must provide. They define the core capabilities needed for hexagonal map generation, graph rewriting, designer configuration and deterministic output.

Non-functional requirements describe quality constraints that influence the design of the system, such as usability, maintainability, performance, extensibility and scope control.

Because the project is a proof of concept, the requirements prioritize configurability and demonstrability over production-ready completeness. The goal is not to build a universal procedural generation package, but to design a focused framework that proves the feasibility of designer-configurable hexagonal map generation in Unity.

ID	Requirement
1	The framework shall produce a game map as a graph of hexagonal cells.
2	The framework shall allow designers to define map generation rules based on global or local cell patterns.
3	The framework shall allow designers to define which pattern matches are considered valid candidates for rule transformations.
4	The framework shall allow rules with valid matches to transform the grid during map generation.
5	The framework shall expose generation configuration inside the Unity editor Inspector window.
6	The framework shall allow map generation without requiring designers to write code.
7	The framework shall support repeatable generation by producing the same result from the same configuration and seed.
8	The framework shall provide visual feedback for generated hexagonal maps inside the Unity editor scene view.

Table 7: Functional requirements for the proposed framework

ID	Requirement
1	The framework should be understandable for users with limited programming knowledge.
2	The framework should keep generation configuration visible and editable through Unity editor interfaces where possible.
3	The framework should be modular enough to separate grid representation, rule definition, rule matching, rule application, and visualization.
4	The framework should be maintainable by keeping generation logic independent from presentation logic.
5	The framework should be extensible enough to support additional rule types, cell properties, or map features in future work.
6	The framework should generate small and medium-sized proof-of-concept maps within an acceptable time for interactive editor use.
7	The framework should make generated results reproducible for testing, debugging, and comparison.
8	The framework should remain focused on hexagonal grid-based map generation and avoid expanding into a general purpose procedural generation system.
9	The framework should rely on Unity native features where possible to reduce external dependencies and improve accessibility.

Table 8: Non-functional requirements for the proposed framework

3.2 Architecture

The functional and non-functional requirements led to an architecture centered on a modular Unity framework rather than a standalone application or external generation service. Since the goal of the project is to demonstrate designer-configurable hexagonal map generation inside Unity, the system was designed as a framework integrated into the editor, composed of:

- Unity MonoBehaviour orchestration scripts.
- ScriptableObject configuration assets.
- Graph-based generation core.
- Rule rewriting subsystem.
- Materialization layer that converts abstract generated data into visible scene objects.

The requirements influenced the architecture in four main ways. First, the need to represent maps as hexagonal cell graphs required a dedicated graph core that stores grid structure, cell identity, adjacency and properties separately from visual objects. Second, designer-configurable generation required rule structure, definitions for how rules find valid matches for execution, prerequisite conditions for rule and rewrite actions that mutate the grid have a need to be represented as Unity assets editable in the Inspector. Third, the need for reproducible generation required a deterministic seed and random stream subsystem. Lastly, the need for visual feedback inside Unity required a separate

materialization layer that translates the generated graph into prefab instances in the scene view.

3.2.1 Framework-specific terminology

Before discussing how the requirements influenced the architecture, it is useful to define several terms used throughout the framework. The system represents a generated map as an abstract grid of hexagonal cells. Each cell has a position in the grid and an associated property, such as a terrain or region type. The generation process modifies these properties over time, and the final grid is later converted into visible Unity objects.

Rulebook:

A rulebook is the main designer-facing generation configuration. It contains an ordered list of rule entries that are executed during generation. Each rule entry describes one possible transformation of the grid. A rule entry does not directly contain all generation logic itself. Instead it combines smaller configurable parts such as a match definition, prerequisite conditions, a match selection policy and a rewrite action.

Match definition:

A match definition describes the cell pattern that a rule is looking for. Pattern matching is the process of scanning the grid and finding cells or regions that satisfy this definition. For example, a match may require an anchor cell (cell that is being considered for a match) with a specific property, a cell adjacent to another property or a cell inside an active subgraph. The result of this process is a set of candidate matches.

Match selection policy:

A match selection policy determines which valid candidate matches are used by a rule entry. The framework supports policies such as selecting the first valid anchor, selecting a random valid anchor or applying the rule to all valid anchors. This makes rule behavior configurable without changing code and allows designers to choose between deterministic, random and expansive transformations.

Prerequisite:

A prerequisite is an additional condition that must be satisfied before a rule entry can apply its transformation. While a match definition usually describes a local or structural pattern, prerequisites can express broader conditions, such as counts for properties present on the grid, property grid coverage, adjacent property pairs or subgraph selection constraints. This separation allows the framework to distinguish between finding possible locations and deciding whether the rule should run at all.

Rewrite action:

A rewrite action performs the actual modification once a valid match has been selected. Actions can replace an anchor property, place a shapes at an anchor, draw a path between properties, flood fill a subgraph or modify a subgraph in other ways. Actions do not directly manipulate the grid data structure. They use a mutation gateway, which provides controlled operations for painting cells, shapes, paths and subgraph regions.

Subgraph:

A subgraph is a region of the grid used to scope generation. Some rules operate over the entire grid, while others operate only within an active subgraph which represents only a part of the whole grid. Subgraphs allow the framework to express higher-level map structures and then apply local transformations inside those structures.

Materialization:

Materialization is the process of converting the abstract generated grid into visible Unity scene objects. The rewrite system operates on cell properties rather than prefabs. After generation is complete, the materialization layer reads the grid and instantiates the appropriate tile prefabs based on the generated properties. This keeps procedural logic separate from visual presentation.

Requirement driver	Architectural response	Reason
Maps must be represented as hexagonalcell graphs.	A graph core stores grid dimensions, cells, adjacency and cell properties.	This keeps generated maps independent from Unity scene objects.
Designers must define generation rules based on patterns.	Rulebooks contain rule entries composed of match definitions, prerequisites and rewrite actions.	Generation behavior can be configured as data instead of being hard-coded.
Designers must control which matches are valid.	Matching, prerequisite condition for rule evaluation and match selection are separate stages.	This makes candidate selection explicit and easier to extend.
Rules must transform the grid.	All changes pass through a mutation gateway and specialized grid mutation services.	Centralized mutation prevents uncontrolled grid changes and gives actions a stable API.
Configuration must be available in the Unity Inspector.	Topology, properties, rulebooks, match definitions, actions and materialization settings are Unity assets.	Unity native assets are familiar to designers and avoid custom external tools.
Designers should not need to write code.	Generation behavior is assembled from reusable rule, match, prerequisite and action assets.	Designers can combine existing components through the Inspector.
Output must be repeatable from the same seed.	The framework uses deterministic seed normalization and separate random streams.	This makes generated results reproducible for testing and debugging.
Generated maps need visual feedback.	A materialization layer converts the generated graph into prefab instances.	The same abstract map can be inspected visually without coupling rules to rendering.
The system should be understandable to non-programmers.	The architecture favors visible assets, Inspector fields and named rule components.	Users can reason about configured generation steps instead of source code.
Generation logic should remain separate from presentation logic.	The graph and rewrite layers are separate from the materialization layer.	Rules operate on abstract properties while visualization remains replaceable.
The framework should be extensible.	Core behavior is split into match definitions, prerequisites, rewrite actions, contexts and mutation services.	New rule types or map features can be added without rewriting the engine.
Small and medium maps should generate interactively.	Generation runs in-process using arrays, dictionaries and bounded rewrite counts.	Simple data structures support acceptable proof-of-concept editor performance.

Requirement driver	Architectural response	Reason
The project should remain focused.	The architecture targets hexagonal grid rewriting only.	This avoids unnecessary complexity from a general-purpose procedural generation system.
Unity native features should be used.	The framework uses C#, MonoBehaviours, ScriptableObjects, prefabs, the Inspector and Unity logging.	This reduces dependencies and keeps the workflow accessible inside Unity.

3.3 System diagrams

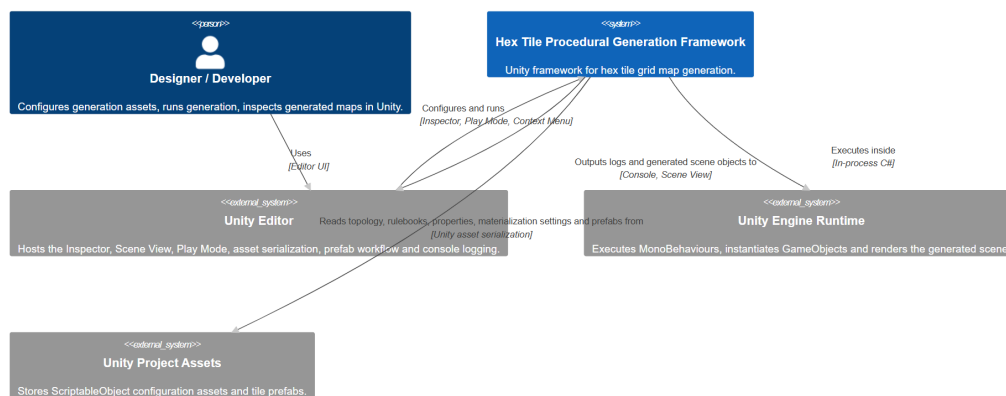


Figure 11: C4 Level 1 system context.

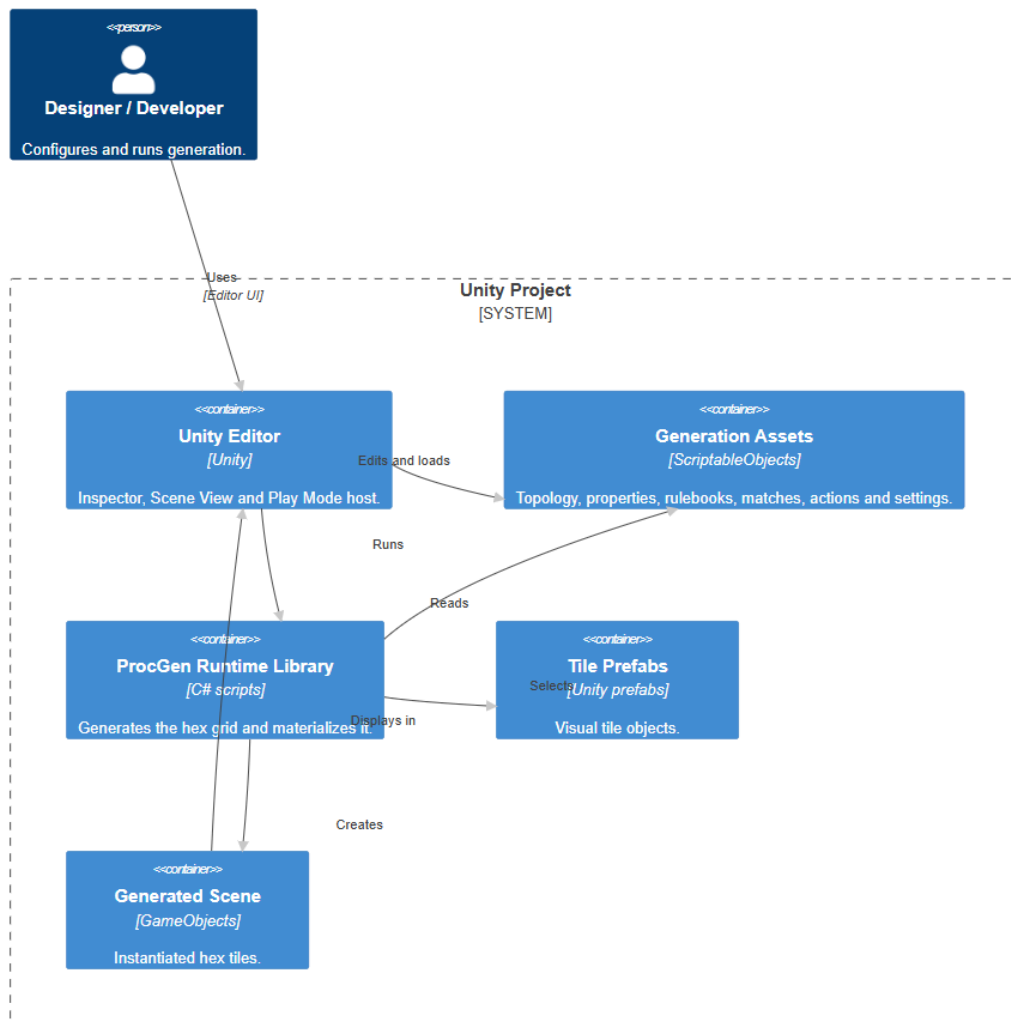


Figure 12: C4 Level 2 container diagram.

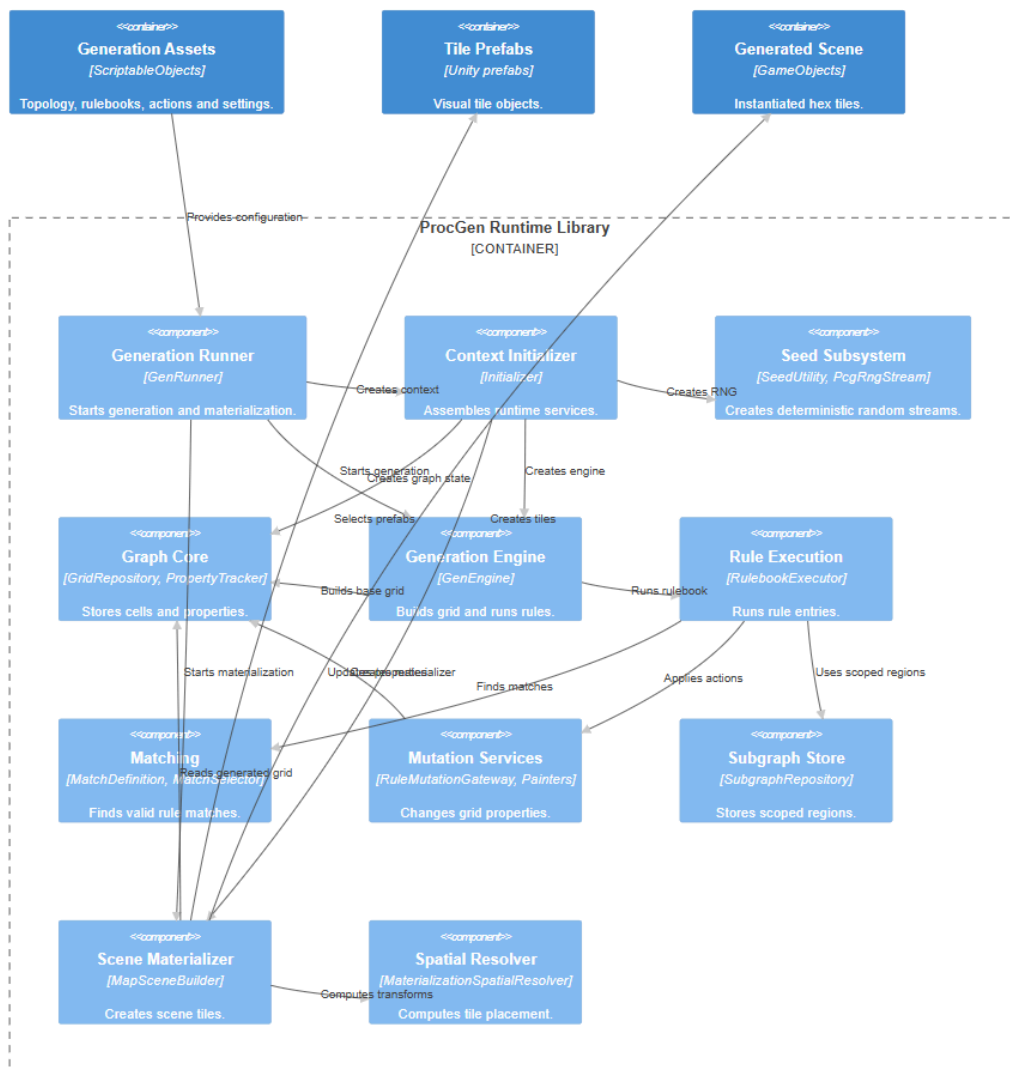


Figure 13: C4 Level 3 component diagram.

3.4 Technology stack

The framework is built as a Unity-native proof of concept. The main programming language is **C#**, and the system runs inside the Unity Editor and Unity Engine runtime. Unity MonoBehaviour scripts are used for scene-level orchestration, Unity ScriptableObject assets are used to store designer-editable configuration such as graph topology, cell properties, rulebooks, match definitions, rewrite actions and materialization settings.

The generated map is represented with custom in-memory data structures, mainly grid arrays, dictionaries and lightweight cell reference objects. The framework does not use an external database, server backend or messaging system. Visual output is produced through Unity prefabs, which are instantiated into the scene by the materialization layer. Deterministic generation is supported by a custom seed utility and PCG-based random number stream implementation.

3.5 Security

Because the framework is a local Unity editor tool and does not expose a network API, store user accounts or process sensitive data, possible security threats are limited. The main relevant concern is supply-chain risk. The project depends on Unity and Unity packages imported into the project. A compromised package, asset, editor extension or dependency could introduce malicious code into the Unity project or affect generated outputs.

This risk can be reduced by keeping dependencies minimal, preferring Unity native functionality where possible, importing packages only from trusted sources, reviewing third-party code before use and keeping Unity/editor dependencies under version control where appropriate. Since the framework currently relies mostly on custom **C#** code and Unity native features, its supply-chain exposure is lower than a system that depends on many external libraries or services.

4 | Implementation

4.1 Methodology

4.2 Prototyping

4.3 Project standards

4.3.1 Core component implementation

4.4 Testing

4.5 Performance bottlenecks and optimizations

4.6 Deployment

4.7 Documentation

5 | Validation

5.1 Testing approaches for functional requirements

5.2 Testing approaches

5.3 Test cases

5.4 System limitations

6 | Conclusion

- 6.1 Thesis summary**
- 6.2 Alignment with objectives**
- 6.3 Challenges**
- 6.4 Limitations**
- 6.5 Future work**

Glossary

Bibliography